AEP FROM ZERO • PART 4 • MULTIPLY THE TEAM (AI)

EPISODE 34

RAG Explained Simply: AI Chatbot on Your Data

RAG explained simply: what retrieval-augmented generation is, and how to build an AI chatbot on your own company data that cites its sources and admits what it does not know. You will learn LLM basics for beginners (tokens, context windows, embeddings and temperature), the RAG pipeline (chunk, embed, store in a vector store, retrieve), and RAG vs fine-tuning for company knowledge. You will also learn a chunking strategy for RAG, why an AI chatbot gives fluent but wrong answers, how citations and

Previously: check the AI, every time

- Last time: ten questions scored against the truth
- Two subject lines kept, three rejected with reasons
- A caption corrected: path views are not people
- Today: Anjali wants a chatbot over Bloom's content
- Goal: cite every answer, admit what it does not know

KEY IDEA

Kavya learned to verify Adobe's built-in AI. Now Anjali wants a chatbot over Bloom's own products and campaigns, built the honest way.

Why not just ask a general chatbot?

- A general chatbot never read Bloom's catalogue
- Asked anyway, it may invent a price or an offer
- Hallucination: a fluent, confident, wrong answer
- Built outside the Adobe UI, on Bloom's own content
- Honest version: cite sources, refuse when unsure

KEY IDEA

A general chatbot has never read Bloom's catalogue or campaign FAQ. Asked about them, it may still answer fluently and invent the facts.

LLM basics: tokens, context, temperature

Term	Plain meaning
Large language model	Predicts the next words from text
Token	A word, or a piece of a word
Context window	How many tokens it can hold at once
Temperature	How adventurous its word choice is
Cost and latency	More tokens: more money, more time

KEY IDEA

A large language model reads tokens, holds only so many in its context window, and temperature sets how adventurous its word choices are.

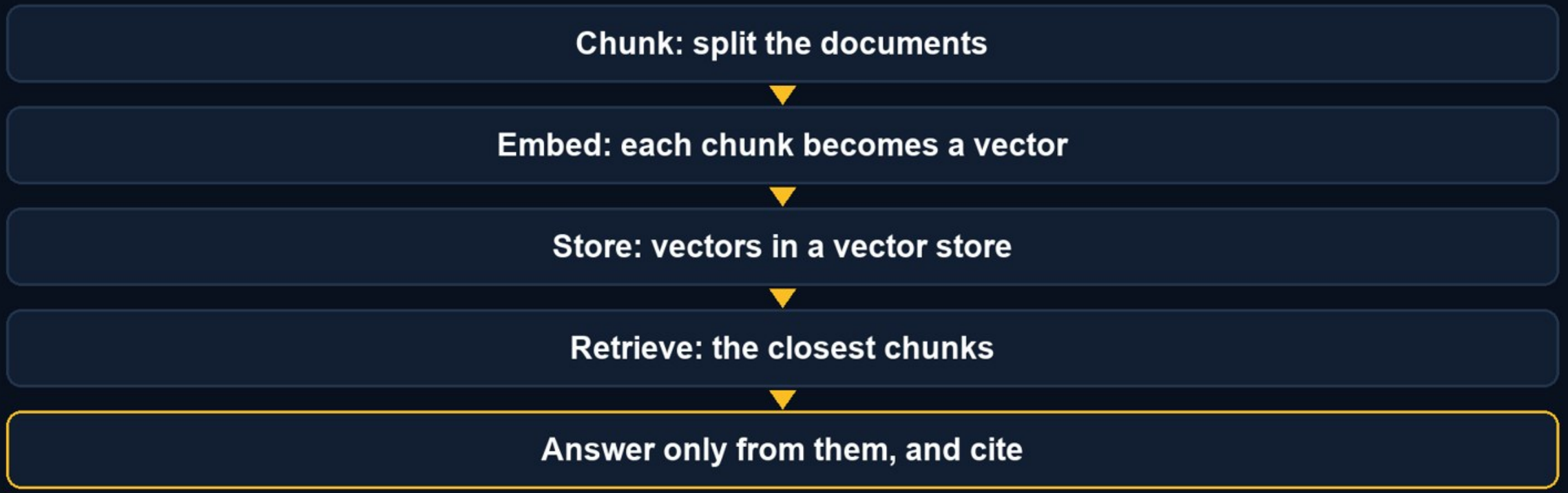
Embeddings: meaning turned into numbers

- Embedding: text turned into a list of numbers
- That list of numbers is called a vector
- Similar meanings sit close together
- A how-much question finds the price line
- Search by meaning, not by matching words

KEY IDEA

An embedding turns a passage into a list of numbers, a vector, so passages with similar meanings sit close together even when the words differ.

What is RAG? Four moves, in simple words



KEY IDEA

Retrieval-augmented generation fetches the relevant passages first, then lets the model answer only from them: chunk, embed, store, retrieve.

RAG versus fine-tuning for company knowledge

Question	Fine-tuning	RAG
What changes?	The model's habits	What the model reads
A price changes	Train the model again	Update one chunk
Can it cite?	No page to point at	Cites the chunk
Not in the data?	May guess from habit	Can refuse honestly

KEY IDEA

Fine-tuning changes the model's habits. RAG hands it today's documents. For facts that keep changing, like prices and offers, RAG wins.

The golden set, written before the build

Type	How many	Example
Answerable	5	What does the serum cost?
Tricky	3	Worded unlike the catalogue
Unanswerable	2	When is the next sale?
Target	At least 8 of 10	Cite every answer, or refuse

KEY IDEA

Ten real questions with known answers, written first: five answerable, three tricky, two unanswerable. The target is at least eight of ten.

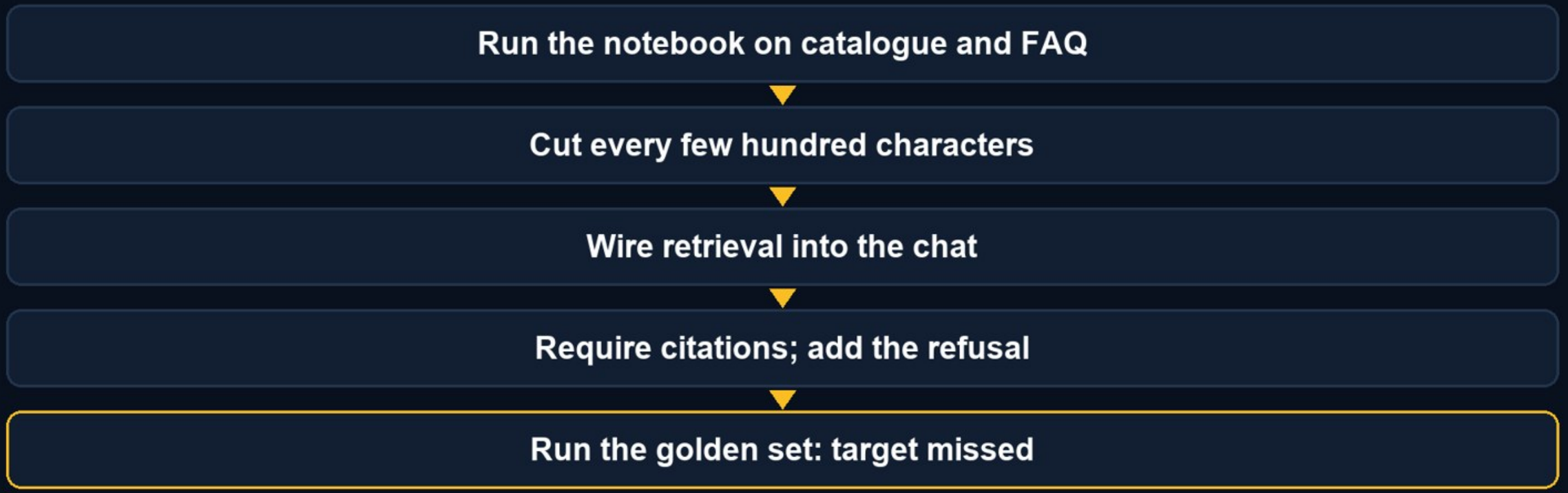
Stop hallucinations: cite it or refuse

- Instruction: answer only from the retrieved chunks
- Every answer cites the chunk it used
- Weak retrieval: say so, never guess
- Citations make human review practical
- Before launch: ask what your documents lack

KEY IDEA

Answer only from retrieved chunks, cite the chunk every time, and when retrieval is weak, say so instead of guessing.

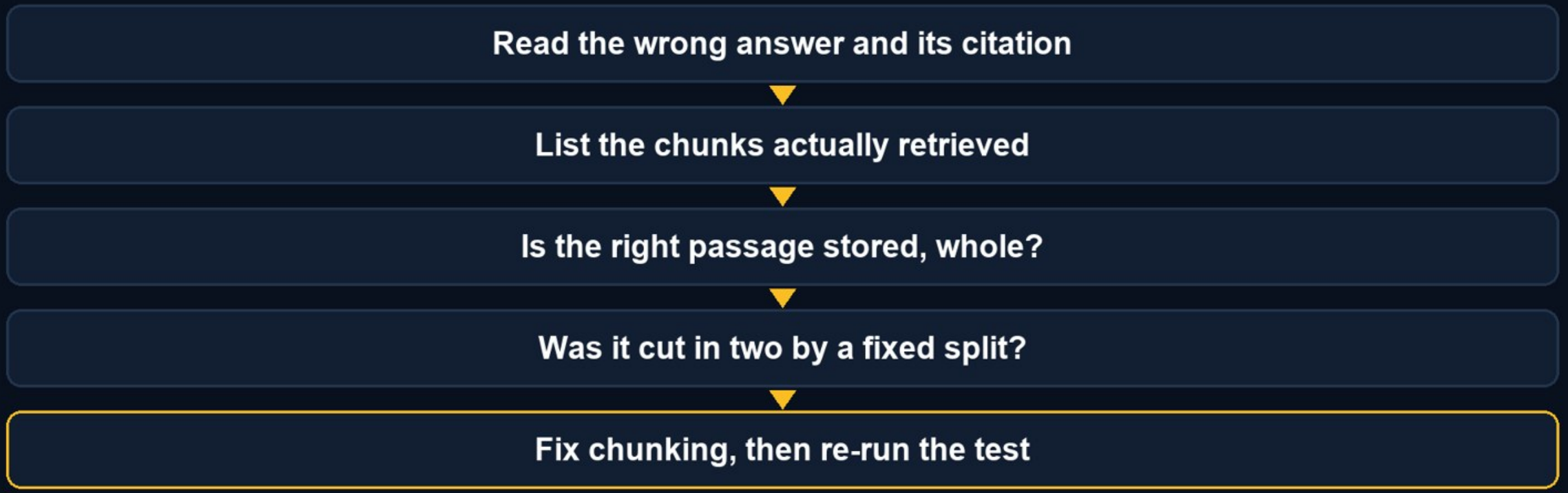
Inside the build: the first index



KEY IDEA

The first index cut the catalogue and FAQ every few hundred characters. The golden set ran, and the first run missed the target.

Fluent but wrong? Check retrieval first



KEY IDEA

Fluent but wrong almost always means the right passage never reached the model. Check what was retrieved before you touch the model.

Chunking decides quality: cut on structure

Check	Fixed character cut	Headings and sections
Where it splits	Wherever the count ends	At the document's own breaks
One product	Can land in two chunks	One product, one chunk
One FAQ answer	Can lose its question	Question and answer together
What comes back	Partial matches	Whole, citable passages

KEY IDEA

Too small and a chunk loses its context; too large and the key sentence drowns. Split on headings and sections, not a character count.

The second run, and where the value sits

- Same ten questions, nothing edited to fit
- At least 8 of 10, every answer cited
- Both unanswerable questions refused honestly
- Product and campaign documents only, no customer data
- The model is a commodity; the content is not

KEY IDEA

Same golden set, new chunks: at least eight of ten with citations, both unanswerable questions refused. The model is a commodity; the content is not.

Interview corner

- Fluent but wrong: check retrieval first
- Golden set written before the build
- Chunk on headings and sections
- Cite every answer; refuse when retrieval is weak
- Fine-tuning changes how it talks; RAG, what it reads

KEY IDEA

Say it in one sentence: fluent but wrong almost always means the right source never reached the model. Then back it with your build.

Recap, your question, and agents

- Tokens, context window, embeddings, temperature
- RAG: chunk, embed, store, retrieve, then answer
- Chunk on structure; cite or refuse
- Golden set first; fluent but wrong means retrieval
- Next: agents, tool calling, MCP and the brake

KEY IDEA

Retrieve first, answer only from it, cite or refuse, and test with a golden set. Next: an AI that does the legwork, with the brake built first.